

Proyecto 2: Perceptrón.

Control de temperatura por medio del perceptrón.

Hurtado Gallardo María Teresa

Universidad Nacional Autónoma de México.
Facultad de Estudios Superiores Cuautitlán.
Lunes 6 de Octubre del 2025.

Abstract. El presente reporte compila el procedimiento, entrenamiento y desarrollo de un control de temperatura haciendo uso del perceptrón que clasifica esta medición y arroja una salida visual, en este caso LEDs. Las tres categorías posibles para la temperatura medida son: frío, templado o caliente. Los resultados muestran una acertada capacidad de clasificar temperaturas tanto dentro como fuera del rango de entrenamiento del perceptrón.

Keywords: Perceptrón, Temperatura, Raspberry pi pico w, Sistema de control.

1 Introducción

Un control de temperatura se trata de un sistema que monitorea constantemente y, en algunos casos, ajusta la temperatura de un espacio o proceso de acuerdo a una configuración dada.

Estos controles son necesarios y especialmente útiles en ambientes donde se manejan elementos o sustancias sensibles a los cambios en esta, por ejemplo, alimentos perecederos, combustibles, químicos, siembras, etc. En casos específicos como en el manejo de combustibles los cambios de temperatura representan un peligro no solo a nivel económico, pues se puede desaprovechar por completo el producto, sino a nivel de seguridad, pues puede provocar accidentes graves y pérdidas importantes a muchos niveles.

Los sistemas de este tipo clasifican las temperaturas para poder mantener el espacio en un umbral aceptable para el trabajo que se está desarrollando, es por eso que se eligió la realización de un control de este tipo, que mediante el uso del perceptrón para encasillar en 3 categorías la temperatura del ambiente, en frío, templado y caliente.

En este trabajo se documentará el proceso realizado para desarrollar un control de temperatura que tenga interacción en tiempo real con el ambiente y salidas físicas características para la categorización de la temperatura.

2 Conocimientos previos

A los algoritmos de inteligencia artificial en la actualidad los rige la unidad primaria del perceptrón, el cual se trata de un modelo matemático que imita el comportamiento de una neurona biológica, utilizando sus redes y correlaciones para generar salidas a partir de entradas determinadas.

Para entender el comportamiento del perceptron hay que entender primero a la neurona, se trata de una célula que constituye la unidad anatómico funcional del sistema nervioso cuya principal función es recibir, procesar y transmitir información a través de señales químicas y eléctricas gracias a la excitabilidad eléctrica de su membrana plasmática. Están especializadas en la recepción de estímulos y conducción del impulso nervioso (en forma de potencial de acción) entre ellas mediante conexiones llamadas sinapsis, o con otros tipos de células como, por ejemplo, las fibras musculares. Altamente diferenciadas, la mayoría de las neuronas no se dividen una vez alcanzada su madurez; no obstante, una minoría sí lo hace.[1]

Las partes y estructura de la neurona multipolar típica, el principal tipo neuronal que se encuentra en el sistema nervioso central, que puede ser vista de forma gráfica en la figura 1, se compone de lo siguiente:

- Parte receptora (dendritas): recibe y conduce las señales eléctricas hacia el cuerpo celular.[2]
- Parte integradora (normalmente equiparada al cuerpo celular/soma): contiene el núcleo y la mayoría de los orgánulos de la célula, actuando como el centro trófico de toda la neurona. Aquí es donde se procesan e integran las señales de entrada para determinar si se generará un impulso eléctrico (potencial de acción).[2]
- Parte conductora (axón): conduce los impulsos eléctricos lejos del cuerpo celular (soma).[2]

- Parte transmisora (botones terminales del axón): donde los axones se comunican con otras neuronas o efectores (estructuras diana que responden a los impulsos nerviosos). [2]

La base de la función neurológica radica en las neuronas como entidades discretas, cuya interacción, mediada por sinapsis, conduce a la aparición de respuestas complejas. [1]

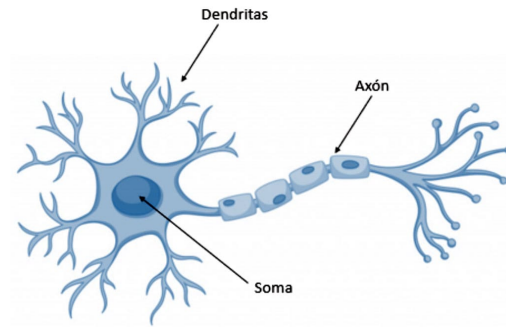


Fig. 1. La neurona y partes que la componen [3]

Ahora bien, el modelo matemático desarrollado a partir del análisis del comportamiento de esta célula se trata del perceptrón, la neurona artificial, un algoritmo capaz de generar un criterio para seleccionar un subgrupo a partir de un grupo de componentes más grande.[5]

En el modelo hay entradas, pesos sinápticos (relacionados cada uno a una entrada), una desviación (el bias), una función de activación y una salida; La sumatoria de todas las multiplicaciones entre entradas y pesos sinápticos menos el bias, determina si la función de activación se acciona o no. Se puede describir el comportamiento por medio de las siguientes expresiones.

$$x_1w_1 + x_2w_2 + \dots + x_nw_n - w_o(t) > 0 \text{ --- } out(t) = 1$$

$$x_1w_1 + x_2w_2 + \dots + x_nw_n - w_o(t) < 0 \text{ --- } out(t) = 0$$

El perceptrón y sus elementos se aprecia mas claramente en la figura 2, donde las x corresponden a las entradas, w a los pesos sinápticos, w_o al bias y $out(t)$ a la salida.

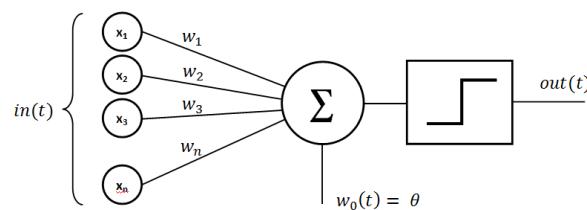


Fig. 2. Perceptrón [4]

Para que esta neurona funcione, debe ser entrenada, con conjuntos de datos relacionados al propósito que cumplirá y el comportamiento esperado que tendrá con respecto a estos, modificando sus pesos y desviación (bias) hasta que responda como se espera. Esto se logra mediante el calculo constante de nuevos valores, y la magnitud del cambio entre el peso inicial y el siguiente después del entrenamiento esta dado por el factor de aprendizaje.

Error: $E = Z_e - Z_o$

Donde:

- E : Error
- Z_e : Salida esperada
- Z_o : Salida obtenida

Nuevo peso: $w_0^+ = w_0 + \lambda x_0 E$

Donde:

- E : Error
- w_0 : Peso inicial
- λ : Factor de aprendizaje
- x_0 : Entrada

Nuevo bias: $b^+ = b + \lambda(-E)$

Donde:

- E : Error
- b : Bias inicial
- λ : Factor de aprendizaje

3 Metodología

Por su compatibilidad con python y la versatilidad de sus aplicaciones, para este proyecto se eligió la Raspberry pi pico w, una tarjeta de desarrollo basada en el microcontrolador RP2040, diseñada para ser una plataforma de desarrollo flexible de bajo costo con interfaz inalámbrica de 2.4 Ghz, con salidas de voltaje de 3.3v y un chip Infineon CYW43439 para implementar la interfaz inalámbrica de 2.4 GHz[6].

Se seleccionó un sensor de presión barométrica y temperatura BMP280, por su tamaño, accesibilidad y bajo consumo de energía que puede ser suministrado desde las salidas de voltaje de la tarjeta seleccionada. Adicionalmente se uso una pantalla oled de 0.96 pulgadas y una resolución de 128 x 64 píxeles SSD1306, mediante el protocolo i2c, para verificar las mediciones de la temperatura en tiempo real.

El hardware seleccionado para el proyecto, puede apreciarse en la figura 3.

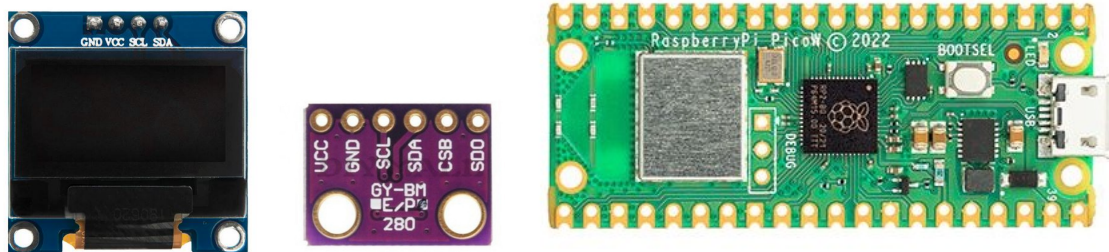


Fig. 3. Pantalla[7], modulo[9] y tarjeta[8] utilizados en el proyecto

El lenguaje python fue seleccionado para desarrollar el programa de este proyecto, en jupyter notebook se desarrolló y entrenó a la neurona y en el entorno de desarrollo visual studio code, se implementó junto con el sensor, la pantalla y se agregaron 3 LEDs, uno para para cada categoría de temperatura.

El primer paso en el programa fue generar la función de activación, con parámetros de entrada, pesos sinápticos y bias.

```
#Funcion de activacion
def act (peso ,x1 ,b):
    z=peso * x1
    sumatoria=z+b
    if sumatoria>0:
        return 1
    else:
        return 0
```

Posteriormente, se generó un vector de entrenamiento con valores de temperaturas en todos los rangos posibles a tomar y dos vectores de salida, uno donde la salida alta sea cuando los valores estén por debajo de los 5°C y otro donde la salida alta sea por encima de los 27°C.


```
peso2 += taza_aprendizaje * Temps[a] * error2
b2 += taza_aprendizaje * error2
print(error_tot2, end="-")
```

El error acumulativo en este entrenamiento a lo largo de las épocas fue el siguiente:

[illegible]

Una vez terminado el entrenamiento se hicieron pruebas con valores ficticios para verificar la respuesta de la función de activación con los pesos para cada vector de salida y se imprimieron los valores finales que adoptaron los pesos.

```
#testeo del entrenamiento con el primer vector
act(peso1, 0.12, b1), act(peso1, -0.2, b1), act(peso1, -0.44, b1), act(peso1, 0.9, b1)

peso1, b1
```

Salidas: (1, 1, 1, 0) (np.float64(-1.41), np.float64(0.22999999999999993))

```
#testeo del entrenamiento con el segundo vector
act(peso2,0.3,b2),act(peso2,0.2,b2),act(peso2,-0.3,b2),act(peso2,0.5,b2)

peso2,b2
```

Salidas: (1, 0, 0, 1) (np.float64(0.26), np.float64(-0.070000000000000015))

Después, en la tarjeta, que sería el interprete para que los valores de temperatura llegasen a la neurona se configuró, tanto el sensor [11] como la pantalla [10] usando sus respectivas librerías mediante el siguiente código.

Para el sensor se usaron los pines GPIO 0 y 1 para SDA y SCL respectivamente, para la pantalla GPIO 4 y 5 igualmente para SDA y SCL y los pines GPIO 13, 14 y 15 para los LEDs. En este código, además, se imprimen las medidas recolectadas al ambiente por el sensor en la consola para verificar coherencia con lo mostrado en la pantalla oled.

```

from lib.bmp280 import BMP280
import lib.ssd1306
import machine
from machine import I2C, Pin, SoftI2C
import time

#INICIALIZACION DE PANTALLA
bus = SoftI2C(scl=Pin(5), sda=Pin(4))
oled_width = 128
oled_height = 64
oled = lib.ssd1306.SSD1306_I2C(oled_width, oled_height, bus)
devices = bus.scan()
if devices:
    print('Pantalla encontrada en la direccion', devices)
    oled.text("INICIANDO...", 20, 30)
    oled.show()
    time.sleep(3)
else:
    print('No se encontraron dispositivos I2C')

#INICIALIZACION DEL SENSOR HUMEDAD Y TEMPERATURA
i2c = I2C(0, scl=Pin(1), sda=Pin(0), freq=40000)
bmp = BMP280(i2c)
bmp.normal_measure()

#LEDS DE SALIDA
led_r=machine.Pin(13,machine.Pin.OUT)
led_v=machine.Pin(14,machine.Pin.OUT)
led_b=machine.Pin(15,machine.Pin.OUT)

devices = i2c.scan()

```

```

if devices:
    print('Sensor encontrado en la direccion', devices)
    oled.fill(0)
    oled.text("SENSOR LISTO", 20, 30)
    oled.show()
    time.sleep(3)
else:
    print('No se encontraron dispositivos I2C')

#CICLO PRINCIPAL
while True:
    #DEFINICION DE VARIABLES PARA DATOS
    oled.fill(0)
    temp_pantalla = bmp.temperature
    pres_pantalla = round(bmp.pressure)

    #IMPRESION DE LOS DATOS EN PANTALLA
    print("*****")
    print("Temperatura: {:.2f}-C".format(temp_pantalla))

    #VALORES EN LA OLED
    oled.text('Temperatura:', 15, 25)
    oled.text("{:.2f}-C".format(temp_pantalla), 35, 35)
    oled.show()

```

Para agregar la neurona hace falta definir, de nuevo una función de activación, relacionarlos al comportamiento de los LEDs e ingresar como valores estáticos los pesos y bias generados en el entrenamiento. Debido a que estas neuronas solo generan 2 categorías posibles para las temperaturas, frio o calor, todas aquellas que no entren en ninguno de los dos grupos pertenecerán a un tercero denominado templado.

```

#pesos para los perceptrones
peso1=-1.41
b1=0.22999999999999993
peso2=0.26
b2=-0.070000000000000015
#funcion de activacion
def act (peso,x1,b):
    z=peso * x1
    sumatoria=z+b
    if sumatoria>0:
        return 1
    else:
        return 0
#funcion valores normales
def norm (e1,e2):
    if (e1==0 and e2==0):
        return 1
    else:
        return 0

```

Luego, dentro del ciclo principal, agregamos a los LEDs con la función de activación como parámetro de encendido o apagado.

```

#ENCENDIDO DE LOS LEDS
led_r.value(act(peso2,(bmp.temperature/(100)),b2))
led_v.value(norm((act(peso2,(bmp.temperature/(100)),b2)),(act(peso1,(bmp.
    temperature/(100)),b1))))
led_b.value(act(peso1,(bmp.temperature/(100)),b1))

```

Una vez listo el código, se realizó la implementación física, (mostrada en la figura 4) las conexiones y la carga del programa a la tarjeta.

Finalmente, las pruebas consistieron en forzar condiciones climáticas distintas a la que se experimentaba, para aumentar o disminuir la temperatura y se verificó el comportamiento del circuito, estas demostraciones se muestran en las figuras 5, 6 y 7.

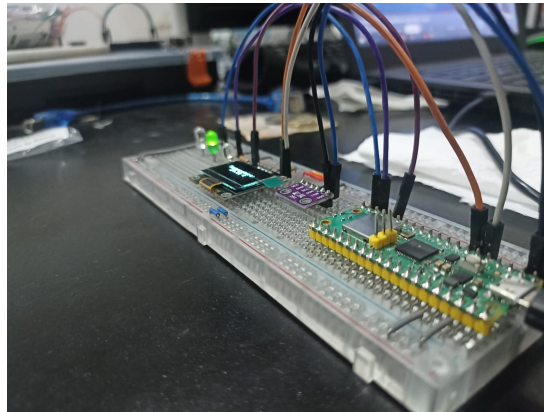


Fig. 4. Circuito armado

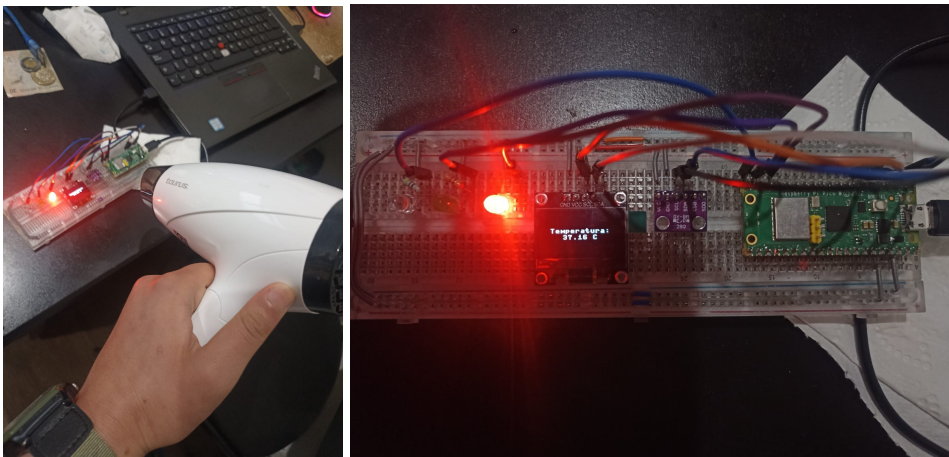


Fig. 5. Respuesta a condiciones de calor

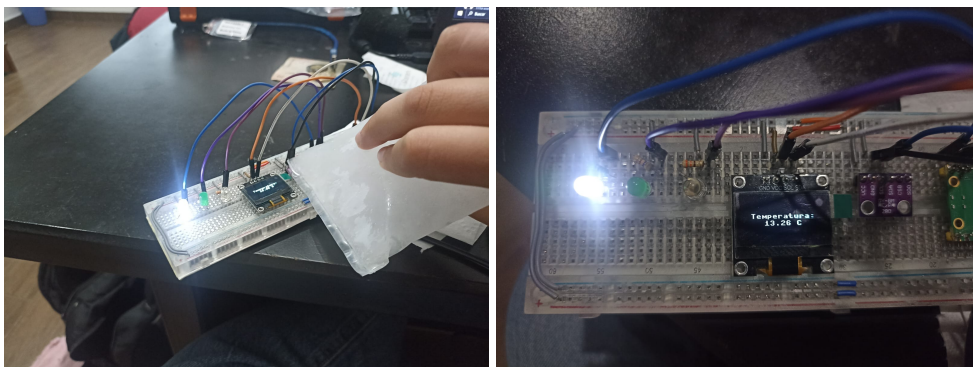


Fig. 6. Respuesta a condiciones de frio

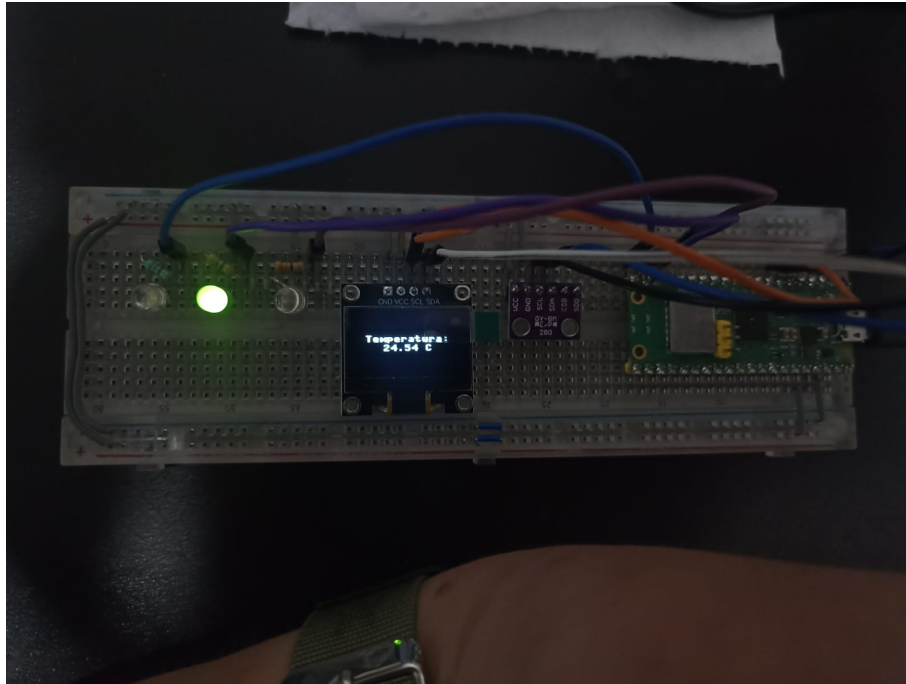


Fig. 7. Respuesta a condiciones de temperatura templada

4 Análisis de resultados

El comportamiento de los LEDs es coherente con el propuesto en los vectores definidos inicialmente para el entrenamiento, incluso con valores no propiamente definidos en el conjunto de temperatura usado para entrenar al perceptrón, el modelo es capaz de clasificar correctamente temperaturas "frías" y "cálidas" de acuerdo con nuestros estándares.

5 Conclusiones

El desarrollo de un sistema de control de temperatura basado en perceptrones, demostró ser una solución eficaz y eficiente para clasificar temperaturas, haciendo uso de un modelo de neurona entrenada con valores de todos los rangos posibles para la clasificación y activando sus salidas correspondientes, en este caso 3 LEDs. Este proyecto también resalta el potencial de los modelos de clasificación y aprendizaje con neuronas artificiales para sistemas de control, debido a su versatilidad y facilidad de aprendizaje frente a cambios ambientales.

References

1. Colaboradores de los proyectos Wikimedia. "Neurona - Wikipedia, la enciclopedia libre". Wikipedia, la enciclopedia libre. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://es.wikipedia.org/wiki/Neurona>
2. "Partes de una neurona". Kenhub. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://www.kenhub.com/es/library/fisiologia/partes-de-una-neurona>
3. "Redes Neuronales artificiales — Blog Xeridia". Xeridia. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://www.xeridia.com/blog/redes-neuronales-artificiales-que-son-y-como-se-entrenan-parte-i>
4. "Perceptrón: ¿qué es y para qué sirve?" DataScientest. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://datascientest.com/es/perceptron-que-es-y-para-que-sirve>
5. Colaboradores de los proyectos Wikimedia. "Perceptrón - Wikipedia, la enciclopedia libre". Wikipedia, la enciclopedia libre. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://es.wikipedia.org/wiki/Perceptrón>
6. "Raspberry Pi Pico W". 330ohms. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://www.330ohms.com/products/raspberry-pi-pico-w>
7. "Pantalla OLED SSD1306 de 0,96 pulgadas I2C 128 x 64 píxeles compatible con Arduino y Raspberry Pi". AZ-Delivery. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://www.az-delivery.de/es/products/0-96zolldisplay>

8. "RASPBERRY-PI-PICO-W". AG — Tienda Virtual. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://agelectronica.com/detalle?busca=RASPBERRY-PI-PICO-W>
9. "Sensor BMP280 Temp Presión Altitud - Tienda Prometec". Tienda Prometec. Accedido el 3 de octubre de 2025. [En línea]. Disponible: <https://store.prometec.net/producto/sensor-bmp280-temp-presion-altitud/>
10. "micropython-ssd1306/ssd1306.py at master · stlehmman/micropython-ssd1306". GitHub. Accedido el 4 de octubre de 2025. [En línea]. Disponible: <https://github.com/stlehmman/micropython-ssd1306/blob/master/ssd1306.py>
11. "micropython-bmp280/bmp280.py at master · dafvid/micropython-bmp280". GitHub. Accedido el 4 de octubre de 2025. [En línea]. Disponible: <https://github.com/dafvid/micropython-bmp280/blob/master/bmp280.py>
12. Walker, J.S.: A primer on wavelet and their scientific applications. 2da. edn. Chapman and Hall/Crc, EUA (2008)
13. Misti, M., Misti, Y., Oppenheim, G., Poggi, J.M.: Wavelet Toolbox for use with Matlab, 2nd edition, Ver. 2.0, Release 5.1, The Mathworks E.U.A. (2000)
14. Chen, Y., Li, J., Wang, J.Z.: Machine Learning and Statistical Modeling Approaches to Image Retrieval. Kluwer Academic Publishers (2004)
15. Baeza-Yates, R., Ribeiro-Neto, B.: Modern Information Retrieval. ACM Press, Addison-Wesley, Edinburgh, England (1999)